

In the meantime, in another session, execute the following statements:

```
-- session 2
forumdb=> BEGIN;
BEGIN
forumdb=> SELECT txid_current(), txid_current_snapshot();
  txid_current | txid_current_snapshot
-----+-----
      4931 | 4930:4930:
(1 row)

forumdb=> UPDATE tags SET tag = lower( tag );
-- LOCKED!!!!
```

Transaction 4931 is locked because PostgreSQL cannot decide which data manipulation to apply. On one hand, transaction 4930 applies uppercase to all the tags, but at the same time, transaction 4931 applies lowercase to the very same data.

Since the two changes conflict, and the final result (that is, the result that will be consolidated in the database) depends on the exact order in which changes will be applied (and in particular on the last one applied), PostgreSQL cannot allow both transactions to proceed. Therefore, since 4930 applied the changes before 4931, the latter is suspended, waiting for transaction 4930 to complete either with success or failure. As soon as you end the first transaction, the second one will be unblocked (showing the message status for the UPDATE statement):

```
-- session 1
forumdb=> COMMIT;
COMMIT

-- session 2
UPDATE 6
-- unblocked, can proceed further ...
forumdb=>
```

Therefore, MVCC is not a silver bullet against lock usage but allows better concurrency in the overall usage of the database.

MVCC comes at a cost, however: since the system has to maintain different tuple versions depending on the active transactions and their snapshots, the storage will literally grow over the effective size of consolidated data.

To prevent this problem, a specific tool named `VACUUM`, along with its background-running brother `autovacuum`, is in charge of scanning tables (and indexes) for tuple versions that can be thrown away, therefore reclaiming storage space. But when is a tuple version eligible for being destroyed by `VACUUM`? It is when there are no more transactions referencing the tuple `xid` (that is, `xmin`), that is, when the tuple is no longer consolidated.

Transaction isolation levels

In a concurrent database system, you could encounter three different problems:

- **Dirty reads:** A dirty read happens when the database allows a transaction to see work-in-progress data from other not-yet-finished transactions. In other words, data that has not been consolidated is visible to other transactions. No production-ready database allows that, and PostgreSQL is no exception: you are assured your transaction will only perceive data that has been consolidated, and in order to be consolidated, the transactions that created such data must be complete.
- **Unrepeatable reads:** An unrepeatable read happens when the same query, within the same transaction, executed multiple times, perceives a different set of data. This essentially means that the data has changed between two sequential executions of the same query in the same transaction. PostgreSQL does not allow this kind of problem by means of snapshots: every transaction can perceive the snapshot of the data available, depending on specific transaction boundaries.
- **Phantom reads:** A phantom read is somewhat similar to an unrepeatable read, but what changes between the sequential execution of the same query is the size of the result set. This means that the data has not changed, but new data has been “appended” to the last execution result set.

The SQL standard provides four isolation levels that a transaction can adopt to prevent any of the preceding problems:

- **Read uncommitted:** The lowest level possible.
- **Read committed:** The default isolation level in PostgreSQL.

- **Repeatable read:** Useful for long jobs, as the system does not see the effects of concurrent transactions; this offers us the possibility to work on a consistent snapshot during the entire execution of the transaction.
- **Serializable:** The strongest isolation level available.

Each level provides increasing isolation upon the previous level; so, for example, `READ COMMITTED` wraps the behavior of `READ UNCOMMITTED`, `REPEATABLE READ` wraps `READ COMMITTED` (and `READ UNCOMMITTED`), and `SERIALIZABLE` wraps all of the previous levels.

PostgreSQL does not support all the preceding levels, as you will see in detail in the following subsections. You can always specify the isolation level you desire for an explicit transaction at the transaction's beginning; every isolation level has the very same name, as reported in the preceding list. So, for example, the following begins a transaction in `READ COMMITTED` mode:

```
forumdb=> BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
BEGIN
```

You can omit the optional keyword `TRANSACTION`, even if in our opinion this improves readability. It is also possible to explicitly set the transaction isolation level by means of a `SET TRANSACTION` statement. As an example, the following snippet produces the same effects as the preceding one:

```
forumdb=> BEGIN;  
BEGIN  
forumdb=> SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
SET
```

It is important to note that the transaction isolation level cannot be changed once the transaction has started. In order to have an effect, the `SET TRANSACTION` statement must be the very first statement executed in a transaction block. Every subsequent `SET TRANSACTION` statement that changes the already set isolation level will produce a failure and put the transaction in an aborting state; if the subsequent `SET TRANSACTION` does not change the isolation level, it will have no effect and will produce no error.

To better understand this case, the following is an example of an incorrect workflow, where the isolation level is changed after the transaction has already executed a statement, even if it doesn't change any data:

```
forumdb=> BEGIN;  
BEGIN  
forumdb=> SELECT count(*) FROM tags;
```

```
count
-----
      7
(1 row)

-- a query has been executed, the SET TRANSACTION
-- is not anymore the very first command
forumbd=> SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
ERROR:  SET TRANSACTION ISOLATION LEVEL must be called before any query
```

In the following sections, we will discuss every isolation level in detail.

READ UNCOMMITTED

The `READ UNCOMMITTED` isolation level allows a transaction to be subjected to the dirty reads problem, which means it can perceive unconsolidated data from other incomplete transactions.

PostgreSQL does not support this isolation level because, after all, it is not a true isolation level. In fact, `READ UNCOMMITTED` means that there is no isolation at all among transactions, and this is certainly a situation where interleaving data corruption happens.

You can set the isolation level explicitly, but PostgreSQL will ignore your request and will set it silently to the most robust `READ COMMITTED` one.

READ COMMITTED

The isolation level `READ COMMITTED` is the default one used by PostgreSQL; if you don't set a level, every transaction (implicit or explicit) will have this isolation level.

This level prevents dirty reads and allows the current transaction to see all the already consolidated data every time a single statement in the transaction is executed. We have already seen this behavior in practice in the concurrent session example.

REPEATABLE READ

The `REPEATABLE READ` isolation level imposes that every statement in a transaction will perceive only data already consolidated at the time the transaction started or, more ideally, at the time the first statement of the transaction started.

SERIALIZABLE

The **SERIALIZABLE** isolation level imposes the **REPEATABLE READ** level and ensures that two concurrent transactions will be able to successfully complete, but only if the end result would have been the same if the two transactions ran in sequential order.

In other words, if two (or more) transactions have the **SERIALIZABLE** isolation level and try to modify the same subset of data in a conflicting way, PostgreSQL will ensure that only one transaction can complete and make the other fail.

Let's see this in action by creating an initial transaction and modifying a subset of data:

```
-- session 1
forumdb=> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN
forumdb=> UPDATE tags SET tag = lower( tag );
UPDATE 7
```

To simulate concurrency, let's pause this transaction and open a new one in another session, applying other changes to the same set of data:

```
-- session 2
forumdb=> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN
forumdb=> UPDATE tags SET tag = '[' || tag || ']';
-- blocked
```

Since the manipulated set of data is the same, the second transaction is locked, as we saw in another example before. Now, assume the first transaction completes successfully:

```
-- session 1
forumdb=> COMMIT;
COMMIT
```

PostgreSQL realizes that also making the other transaction able to proceed would break the **SERIALIZABLE** promise because applying the transaction sequentially would produce different results, depending on their order.

Therefore, as soon as the first transaction commits, the second one is automatically aborted with a serializable error:

```
-- session 2
forumdb=> UPDATE tags SET tag = '[' || tag || ']';
ERROR:  could not serialize access due to concurrent update
```

What happens if the transaction manipulates data that apparently is not related? One transaction may fail again; in fact, let's modify one single tuple from one transaction:

```
-- session 1
forumdb=> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN
forumdb=> UPDATE tags SET tag = '{' || tag || '}' WHERE tag = 'java';
UPDATE 1
```

In the meantime, modify exactly one other transaction from another session:

```
-- session 2
forumdb=> BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN
forumdb=> UPDATE tags SET tag = '[' || tag || ']' WHERE tag = 'perl';
UPDATE 1
```

This time, there is no locking of the second transaction because the touched tuples are completely different. However, as soon as the first transaction executes a COMMIT, the second transaction is no longer able to COMMIT by itself:

```
-- session 2 (assume session 1 has issued COMMIT)
forumdb=> COMMIT;
ERROR:  could not serialize access due to read/write dependencies among
transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during commit
attempt.
HINT:  The transaction might succeed if retried.
```

This is quite a common scenario when using serializable transactions: the application or user must be ready to execute their transaction over and over because PostgreSQL could make it fail, due to the serializability of the workflows.

Explaining MVCC

`xmin` is only a part of the story of managing MVCC. PostgreSQL labels every tuple in the database with four different fields, named `xmin` (already described), `xmax`, `cmin`, and `cmax`. Similar to what you learned about `xmin`, in order to make those fields appear in a query result, you need to explicitly reference them – for instance:

```

forumbd=> SELECT xmin, xmax, cmin, cmax, * FROM tags ORDER BY tag;
  xmin | xmax | cmin | cmax | pk | tag | parent
-----+-----+-----+-----+---+----+-----
 4854 |    0 |    0 |    0 | 24 | c++ |
 4853 |    0 |    0 |    0 | 23 | java |
 4852 |    0 |    0 |    0 | 22 | perl |
 4855 |    0 |    0 |    0 | 25 | unix |
(4 rows)

```

The meaning of `xmin` has been already described in a previous section: it indicates the transaction identifier of the transaction that created the tuple. The `xmax` field, on the other hand, indicates the `xid` of the transaction that invalidated the tuple, for example, because it has deleted the data. The `cmin` and `cmax` fields indicate respectively the command identifiers that created and invalidated the tuple within the same transaction (PostgreSQL numbers every statement within a transaction, starting from 0).

Why is it important to keep track of the statement identifier (`cmin`, `cmax`)? Since the lowest isolation level that PostgreSQL applies is `READ COMMITTED`, every single statement (that is, command) in a transaction must see the snapshot of the data consolidated when the command is started.

You can see the usage of `cmin` and `cmax` within the same transaction in the following example. First of all, we begin an explicit transaction, and then we insert a couple of tuples with two different `INSERT` statements; this means that the created tuples will have a different `cmin`:

```

forumbd=> BEGIN;
BEGIN

forumbd=> SELECT xmin, xmax, cmin, cmax, tag, txid_current()
          FROM tags ORDER BY tag;

  xmin | xmax | cmin | cmax | tag | txid_current

```

```

-----+-----+-----+-----+-----+-----
4854 | 0 | 0 | 0 | c++ | 4856
4853 | 0 | 0 | 0 | java | 4856
4852 | 0 | 0 | 0 | perl | 4856
4855 | 0 | 0 | 0 | unix | 4856
(4 rows)

-- first writing command (number 0)
forumdb=> INSERT INTO tags( tag ) values( 'raku' );
INSERT 0 1

-- second writing command (number 1)
forumdb=> INSERT INTO tags( tag ) values( 'lua' );
INSERT 0 1

-- fourth command within transaction (number 3)
forumdb=> SELECT xmin, xmax, cmin, cmax, tag, txid_current()
          FROM tags ORDER BY tag;

 xmin | xmax | cmin | cmax | tag | txid_current
-----+-----+-----+-----+-----+-----
4854 | 0 | 0 | 0 | c++ | 4856
4853 | 0 | 0 | 0 | java | 4856
4856 | 0 | 1 | 1 | lua | 4856
4852 | 0 | 0 | 0 | perl | 4856
4856 | 0 | 0 | 0 | raku | 4856
4855 | 0 | 0 | 0 | unix | 4856
(6 rows)

```

So far, within the same transaction, the two new tuples inserted have an `xmin` that is the same as `txid_current()`; obviously, those tuples have been created by the same transaction. However, note how the second tuple, being in the second writing command, has a `cmin` that holds 1 (command counting starts from 0).

Therefore, PostgreSQL knows when every tuple has been created by means of a transaction and a command within that transaction.

Let's move on with our transaction: declare a cursor that holds a query against the tags table, and delete all tuples but two. The transaction session continues as follows:

```

forumdb=> DECLARE tag_cursor CURSOR FOR SELECT xmin, xmax, cmin, cmax,
tag, txid_current() FROM tags ORDER BY tag;
DECLARE CURSOR

forumdb=> DELETE FROM tags WHERE tag NOT IN ( 'perl', 'raku' );
DELETE 4

forumdb=> SELECT xmin, xmax, cmin, cmax, tag, txid_current()
          FROM tags ORDER BY tag;
  xmin | xmax | cmin | cmax | tag  | txid_current
-----+-----+-----+-----+-----+-----
  4852 |    0 |    0 |    0 | perl |          4856
  4856 |    0 |    0 |    0 | raku |          4856
(2 rows)

```

As you can see, the table now holds only two tuples – this is the expected behavior after all.

However, the cursor has started before the DELETE statement, and therefore, it must perceive the data as it was before the DELETE statement. In fact, if we ask the cursor what data it can obtain, we see that it returns all the tuples as they were before the DELETE statement:

```

forumdb=> FETCH ALL FROM tag_cursor;
  xmin | xmax | cmin | cmax | tag  | txid_current
-----+-----+-----+-----+-----+-----
  4854 | 4856 |    2 |    2 | c++  |          4856
  4853 | 4856 |    2 |    2 | java |          4856
  4856 | 4856 |    0 |    0 | lua  |          4856
  4852 |    0 |    0 |    0 | perl |          4856
  4856 |    0 |    0 |    0 | raku |          4856
  4855 | 4856 |    2 |    2 | unix |          4856
(6 rows)

```

There is an important thing to note: every deleted tuple has a value in xmax that holds the current transaction identifier (4856), meaning that this very transaction has deleted the tuples. However, the transaction has not committed yet; therefore, the tuples are still there but are marked to be tied to the snapshot that ends in 4856.

Moreover, the deleted tuples have a `cmax` that holds the value 2, which means that the tuples have been deleted from the third writing command in the transaction.

Since the cursor has been defined before the statement, it is able to “see” the tuples as they were, even if PostgreSQL knows exactly from which point in time they have disappeared.



Readers may have noted that `cmin` and `cmax` hold the same value, and that is due to the fact that the fields overlap the very same storage.

In the following section, you will see how to disassemble a transaction into smaller pieces by means of savepoints.

Savepoints

A transaction is a block of work that must either succeed or fail as a whole. A savepoint is a way to split a transaction into smaller blocks that can be rolled back independently of each other. Thanks to savepoints, you can divide a big transaction (one transaction with multiple statements) into smaller chunks, allowing a subset of the bigger transaction to fail without having the overall transaction fail. PostgreSQL does not handle transaction nesting, so you cannot issue a nested set of `BEGIN` or `COMMIT/ROLLBACK` statements. Savepoints, however, allow PostgreSQL to mimic the nesting of transaction blocks.

Savepoints are marked with a mnemonic name, which you can use to commit or roll back. The name must be unique within the transaction, and if you reuse the same over and over, the previous savepoints with the same name will be discarded. Let’s see an example:

```
forumdb=> BEGIN;
BEGIN
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Eclipse IDE' );
INSERT 0 1
forumdb=> SAVEPOINT other_tags;
SAVEPOINT
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Netbeans IDE' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES ( 'Comma IDE' );
INSERT 0 1
forumdb=> ROLLBACK TO SAVEPOINT other_tags;
```